

A Reinforcement Learning Architecture for Spatio-Temporal Reallocation in Smart Container Terminals: Independent Proposal and Acceptance Policies with Counterfactual Cost Learning

Author Name¹

Affiliation, Address email@example.com

Abstract. Container terminals aim at improving the efficiency of yard and gate operations by shaping where and when truck jobs are served. Given that external truck jobs are assigned to a yard block and an appointment time before the truck arrives, the ability to revise those assignments as congestion develops is a critical component for terminal operations. In this study, a reinforcement learning architecture that jointly revisits the yard block and the arrival time of an already announced truck job is proposed. A significant challenge is to cope with the indirect effects of a reallocation: moving one job changes the waiting of the following trucks, the crane work sequence, and the vessel supply flow, so the immediate travel time alone does not reveal whether the action was worthwhile. We employ a counterfactual, simulation-based cost signal to tackle this challenge. Our novel idea is to separate the policy that requests a reallocation from the policy that accepts it on behalf of the destination block or time slot, and to let a central procedure commit only the candidates on which both agree under resource constraints. Each policy is trained on the cost difference between discrete-event simulations that differ in a single action from the same state, whereas inference at operating time uses the trained networks alone. The efficacy of the proposed method is verified by numerical experiments based on a continuous synthetic environment whose terminal size, time-of-day truck arrivals, multiple yard cranes, and vessel call volumes follow ranges reported in the literature. The results indicate that the decision path operates stably throughout continuous operation, that the cost reduction is concentrated in arrival-time adjustment under congested conditions, and that the trained model attains lower cost than rule-based policies matched to the same action range.

Keywords: Smart port · Container terminal · Truck arrival management · Yard block reallocation · Counterfactual learning · Reinforcement learning

1 Introduction

Operational efficiency at a smart port depends not only on equipment automation but also on the ability to reallocate resources from information observed

Fig. 1. Reallocation decision flow and the learning/operation split

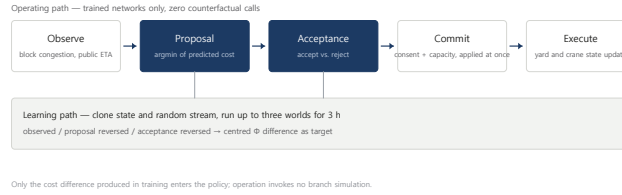


Fig. 1: Reallocation decision flow and the separation of learning from operation. The operating path uses only the trained networks and issues no counterfactual simulation call.

in real time, and smart-port research connects equipment and vehicle information to operating scenarios [1]. At a container terminal the working block and appointment time of an external truck are fixed before arrival, yet during operation demand concentrates on particular slots or blocks and crane queues change with it, so keeping the initial plan makes it difficult to reflect newly observed congestion. This paper revisits both *where* a job is placed and *when* the truck is asked to arrive.

Truck appointment research uses time windows to spread arrival peaks and lets a terminal and carriers negotiate arrival times against their own costs [2, 3], while yard operations research addresses placement, work sequencing, and crane interference [4–6]. The two streams have improved temporal and spatial efficiency separately; this paper places both axes inside one real-time decision. Such a reallocation does not change the cost of a single job alone: when a job moves, the waiting of following trucks, the crane work sequence, and vessel supply flow change as well, so a cost signal including the induced congestion is required, and difference rewards and multi-agent counterfactual baselines provide one by comparing an action with an alternative [7, 8]. Reallocation is also difficult to commit on the proposing side’s benefit alone, since whether the destination can absorb the job must be judged separately; we therefore separate proposal and acceptance and leave consent and feasibility to a central procedure.

We ask whether spatio-temporal reallocation can be composed from independent proposal and acceptance policies, whether counterfactual learning can be separated from operational inference, whether the architecture reduces operating cost, and whether the roles of learning and of the action range can be told apart in the observed change. The contributions are an *independent policy structure*, a *spatio-temporal action range*, *counterfactual cost learning*, a *separation of learning and operation*, and a *decomposable evaluation* that reports execution, action range, and learning effect separately.

2 Related Work

Smart-port research treats automation, intelligence, and environmental design as distinct components of port efficiency [1]. Time-window appointments spread concentrated truck arrivals [2], and negotiation studies show that carriers and a terminal can adjust arrival times while exchanging limited information [3], and arrivals are not uniform over the day [9]. On the spatial side, yard crane research addresses work sequencing, interference, and load balancing [4, 5]; simulation compares how block length and crane deployment affect long-run performance [6]; and the ranking of dispatching rules can reverse with terminal conditions [10]. Reinforcement learning in ports has reached real-time crane dispatching [11]; this paper addresses the layer above it.

Difference rewards isolate the effect of one action from a joint cost by comparing it against a baseline in which other agents keep their actions [7], and counterfactual multi-agent policy gradients construct such a baseline from a centralised critic [8]. We apply this to discrete-event simulation and use the monetary cost difference between two runs differing in a single action as the regression target of each independent policy. A related use of machine learning for resource-level control under environmental uncertainty is found in building demand response [12]; that study likewise combines measured environmental records with synthetic parameters for the variation across individual units and states the synthetic component declaratively, an approach we follow in Sect. 3.1. For policy comparison we use paired runs sharing common random numbers [13].

What distinguishes this work is the combination of an action range covering both arrival-time adjustment and block reallocation, independent proposal and acceptance policies, commitment based on mutual consent, and the separation of counterfactual learning from operational inference within one architecture.

3 Experimental Environment

3.1 A Literature-Calibrated Synthetic Environment

The performance of the proposed architecture is verified using numerical experiments. In order to conduct them in a more realistic setup, scales and operating ranges reported in existing container terminal studies are employed to the extent possible, so the environment is a *literature-calibrated synthetic environment* rather than a replica of a particular terminal. Prior work reports 20 blocks [10], several cranes per block [5], multi-week continuous runs [6], hourly arrival probability [9], three vessel classes with differing call sizes [10], and a triangular range for crane handling time [6]. Within those ranges this study fixed 21 blocks, two cranes per block, 30 days with state carried across the boundary, a base plus three arrival peaks, three vessel classes, and 180 s of crane service time. These fixed values are design choices; the citations establish that they lie within the modelling range of existing work, not that they come from a specific port.

3.2 Terminal and Time Settings

The yard has 21 blocks with two cranes each (42 in total), a block capacity of 1,440 slots at 65% initial occupancy, and a one-dimensional layout with the gate and the quay on opposite sides. The decision interval is 60 s, operation runs for 30 continuous days, and the middle 28 days are compared. The crane work sequence gives priority to service work and, within it, to the shortest expected processing time. A day is a cut at which cost is separated, not a point at which the terminal stops: inventory, queues, crane state, and unfinished jobs carry over, with the first and last day connecting the boundaries.

3.3 External Truck Demand and the Arrival Process

The daily demand was sampled from a discrete distribution over 3,500, 5,000, 7,500, 12,500, and 15,000 trucks with probabilities 0.30, 0.30, 0.25, 0.10, and 0.05. The first three levels represent smooth, normal, and high demand; the last two represent congested and heavily congested conditions. Within a day, the announcement time of each job was drawn from the time-of-day arrival density $f(t)$, a mixture of a uniform base and three normal peaks with (μ_m, σ_m, w_m) equal to (10, 1.5, .317), (15, 2.5, .633), and (21, 1, .05):

$$f(t) = 0.38 U(0, 24) + 0.62 \sum_{m=1}^3 w_m \mathcal{N}_{[0,24)}(t; \mu_m, \sigma_m^2). \quad (1)$$

Time is measured in hours and the truncated density is renormalised over a day. The arrival model follows [9]; the weights and peak centres are design values chosen so that congestion sensitivity can be compared across demand levels, not estimates fitted to a particular terminal. The uniform base keeps arrivals non-zero at night, so the yard state carried across the day boundary is not an artefact of an empty night.

3.4 Vessels and Quay Operations

Vessels were divided into three classes: small (50,000 GT, 3,000 TEU, two quay cranes, KRW 149,500 idle cost per hour), medium (100,000 GT, 7,500 TEU, four, KRW 299,000), and large (150,000 GT, 14,000 TEU, six, KRW 448,500). The call volume of a vessel of capacity C TEU was sampled as $C \cdot u$ with $u \sim U[0.15, 0.30]$, and the daily composition was two small vessels and one medium vessel, to which one large vessel was added with probability 0.30. The throughput of one quay crane was set to 27.5 move/h, within the 24, 30, and 36 move/h range used in the literature [10]. Blocking across quay cranes was summed and divided by the number assigned, giving the idle time per vessel. On the yard side the reference service time of one crane is 180 s per move, so the nominal capacity is $\mu_{YC} = 21 \times 2 \times 3600/180 = 840$ moves per hour. A move is a container movement, not a truck: one truck job may require several moves and rehandles, and the realised service time varies with position and job type, so we report move units and truck units separately.

3.5 Cost Function

The total cost over the interval $[d, d + 1)$ is the sum of four terms:

$$\Phi_d = C_{wait,d} + C_{move,d} + C_{rehandle,d} + C_{vessel,d}. \quad (2)$$

Truck dwell costs KRW 40,000 per hour with the same rate applied again beyond one hour, anchored on the safe trucking freight rate [14]; vessel idling costs KRW 2.99 per GT-hour, converted from the excess berthing rate of KRW 29.9 per 10 GT-hour [15]; additional crane movement (KRW 60,000 per hour) and rehandling (KRW 2,000 each) are design values not taken from the literature. With h_i the dwell time of truck i , m the additional crane working time, R the number of rehandles, and T_v the policy-related idle time of vessel v :

$$C_{wait} = \sum_i 40,000 \{h_i + \max(0, h_i - 1)\}, \quad C_{move} = 60,000 m, \quad (3)$$

$$C_{rehandle} = 2,000 R, \quad C_{vessel} = \sum_v 2.99 GT_v T_v. \quad (4)$$

Structural vessel time that the reallocation policy cannot change, such as berthing and quarantine, is separated as a diagnostic value. All four terms are computed as differences of cumulative values at the same day boundary.

Two properties of the resulting environment were measured over the 30-day training run. First, waiting dominates: C_{wait} accounts for 96.4% of total cost, against 2.2% for rehandling, 1.2% for vessel idling, and 0.1% for additional crane movement. Any conclusion drawn from Φ is therefore governed almost entirely by the safe-freight-rate coefficient taken from regulation [14], and is insensitive to the two coefficients that are design values rather than literature values. Second, both the share of waiting and the service level worsen sharply with demand: waiting rises from 83.2% of cost at 3,500 trucks per day to 91.7% at 7,500 and 99.1% at 15,000, crowding out every other term. Across the month the mean turn time was 0.89 h and the 90th percentile 2.01 h, with 5.0% of 206,904 truck jobs unfinished on the day they were announced; but that 90th percentile ranges from 0.79 h on the lightest days to 7.49 h on the heaviest, a 9.5-fold spread. Congested days are thus not merely more expensive—they are expensive for a different reason, and this is the property the reallocation policy is asked to exploit.

4 Policies and Learning

4.1 Notation, Actions, and Policies

Let the decision interval be $\delta = 60$ s and let the current assignment of job i be $z_i = (b_i, \tau_i)$, with b_i the block and τ_i the announced arrival time. A re-allocation action is $a_i = (b'_i, \Delta_i) \in \mathcal{A}_i(t)$, giving $z_i(a_i) = (b'_i, \tau_i + \Delta_i)$: keeping the original assignment is $a_i^0 = (b_i, 0)$, block reallocation is $b'_i \neq b_i$, and arrival-time adjustment is $\Delta_i > 0$; candidates that change both are allowed. Outbound jobs are excluded from block reallocation because the container to

be retrieved has a fixed location. With n_i the announcement time, g_i the gate-in time, and r_i an indicator of prior review, the set newly reviewed at t is $\mathcal{E}_t = \{i \mid n_i \leq t < n_i + \delta, g_i > t, r_i = 0\}$, so a job that has not yet arrived is reviewed at exactly one time point. The proposal policy selects the action of lowest expected cost from its own block state and the job and action features o_i^P :

$$\hat{a}_i = \arg \min_{a \in \mathcal{A}_i(t)} \hat{c}_\theta^P(o_i^P, a), \quad p_i = \mathbf{1}[\hat{a}_i \neq a_i^0]. \quad (5)$$

If a proposal is made, the destination block or target time slot becomes the accepting party. The acceptance policy compares the expected cost of accepting and rejecting from the proposal message and its own state o_i^A :

$$q_i = \mathbf{1}[\hat{c}_\psi^A(o_i^A, \hat{a}_i, \text{accept}) \leq \hat{c}_\psi^A(o_i^A, \hat{a}_i, \text{reject})]. \quad (6)$$

If the remaining capacity of the target resource is zero, the request is rejected before the network is consulted. The set of mutually agreed candidates is $\mathcal{J}_t = \{i \in \mathcal{E}_t \mid p_i q_i = 1\}$. The problem structure in which two independent parties negotiate arrival times from limited information follows [3].

4.2 Central Commitment Constraints

The central procedure creates no new candidate and commits only within $\mathcal{J}_t$, sorting agreed candidates by the predicted cost of the acceptance policy and breaking ties lexicographically. With $x_i \in \{0, 1\}$ the commitment indicator, $F_b(t)$ the free slots of block b , and $K_k(t)$ the remaining quota of slot k :

$$x_i \leq p_i q_i, \quad \sum_{i \in \mathcal{J}_t} x_i \mathbf{1}[b'_i = b] \leq F_b(t), \quad \sum_{i \in \mathcal{J}_t} x_i \mathbf{1}[k(i) = k] \leq K_k(t), \quad \sum_{t \in \mathcal{T}} \mathbf{1}[i \in \mathcal{E}_t] \leq 1. \quad (7)$$

Block capacity is enforced here; where time-slot quota data are unavailable we set $K_k(t) = \infty$ and read that condition as the range of what time adjustment could provide. All committed assignments are applied at the same instant, so a job committed earlier does not change the state seen by later candidates. Two conditions therefore hold by construction: only candidates carrying both a proposal and an acceptance are committed, and each job is reviewed exactly once.

4.3 Policy Models and Inputs

Both policies are multilayer perceptrons that score fixed-length features per candidate, with one set of weights shared across blocks: 21-dimensional input for the proposal policy and 16 for the acceptance policy, two hidden layers of 64 units with ReLU, and 5,633 and 5,313 parameters respectively—fewer than 11,000 in total. Inputs comprise current waiting, inventory, and remaining crane work,

the public expected arrival time, the route difference, and the announced volume around the target time. Unrealised actual arrival and completion times, and the simultaneous responses of other accepting parties, are excluded.

4.4 Counterfactual Cost Labels

For each sampled decision we clone the state and random stream immediately before branching and run up to three worlds for three hours:

$$\phi_i^F = \Phi_H(\text{observed}), \quad \phi_i^P = \Phi_H(\text{proposal reversed}), \quad \phi_i^A = \Phi_H(\text{acceptance reversed}), \quad (8)$$

with $H = 10,800$ s. Decisions without an acceptance request produce only the proposal comparison. For each policy the two actions are centred on their mean $\bar{\phi}_i^P = (\phi_i^F + \phi_i^P)/2$ and divided by KRW 100,000, giving targets $y_{i,F}^P = (\phi_i^F - \bar{\phi}_i^P)/10^5$ and $y_{i,P}^P = (\phi_i^P - \bar{\phi}_i^P)/10^5$; the same centring is applied to (ϕ_i^F, ϕ_i^A) for the acceptance policy. Subtracting the mean removes the shared cost without changing the order of the two actions. The baseline that reverses a single action connects to difference rewards and multi-agent counterfactual baselines [7, 8].

4.5 Training Settings and Operational Separation

The per-day sample cap is 64 decisions and the exploration probability decreases linearly from 0.50 to 0.05, with an 80/20 train-validation split at the decision level. Optimisation uses Adam (3×10^{-4}), a Huber loss ($\beta = 1.0$), minibatches of up to 256 rows, $\max(50, \min(600, 4 \times \text{rows}))$ updates, and gradients clipped at 1.0. Counterfactual branches run in parallel across CPU cores and each day's labels are replaced after that day's update. At evaluation the weights are frozen, exploration is disabled, and no counterfactual branch is invoked.

Training the pair took 4.7 hours of CPU time over 24 passes. At operating time a decision costs one forward pass per candidate through a network of the size given above, which is negligible beside the constraint check it accompanies; the expense of the method lies entirely in generating the counterfactual labels, and that work is offline and never enters the operating path.

5 Numerical Experiments

5.1 Scope of the Results

The results below come from the middle 28 days of a 30-day continuous diagnostic run generated with random seed 9,900,950, which does not overlap with training. Weights were frozen and the exploration probability set to zero, and all policies on a given day shared the same truck, vessel, and service random streams. Each day stores the four cost terms, the demand level, the transaction types, the unfinished jobs, and the long waits; the source code, the per-policy raw results, and the scripts that generate every figure are released with the paper, so that the seeds, checkpoints, and commands can be replayed.

Table 1: Every policy on the same 28 days, sorted by cost reduction. The two rightmost columns compare each policy with the proposed policy day by day. Reading down the *action range* column shows the pattern: every policy that uses arrival time reduces cost, and every policy restricted to block reallocation sits near zero.

Policy	Action range	Reduction	vs. proposed	
			Days lower	p
Proposed policy	block + time	+14.72%	—	—
Proposed, time only	time	+13.44%	—	—
Untrained model	block + time	+7.97%	18/28	0.185
Rule: least-loaded slot	time	+3.41%	—	—
Rule: least-loaded block + slot	block + time	+1.51%	20/28	0.036
Rule: least slack	block	+0.53%	20/28	0.036
Rule: first-come-first-served	block	+0.17%	22/28	0.004
Rule: shortest processing time	block	+0.05%	21/28	0.013
Rule: net gain	block	−0.48%	21/28	0.013
Proposed, block only	block	−1.31%	—	—
No reallocation	—	0.00%	26/28	<0.001

5.2 Confirmation of Architectural Operation

Six structural conditions were confirmed by different paths. Separate models for proposal and acceptance, the absence of any commitment on one-sided consent, the exclusion of realised future arrival and completion times from policy inputs, and the absence of any state reset at the day boundary follow from the implementation, the commitment constraints, and automated tests. Two were confirmed in the runs themselves: counterfactual simulation calls during frozen-policy evaluation numbered **zero**, and the identity check—a branch that changes no action reproducing the action and cost of the main run—passed 4/4 in the training run, where counterfactual labels exist.

5.3 Paired Daily Comparison Across Policies

The cost difference is the proposed policy minus the comparison policy; a negative value means the proposed policy costs less. Days are treated as independent comparison units and a two-sided sign test is used (Table 1). A Wilcoxon signed-rank test on the same pairs, which uses the size of each daily difference and not only its sign, reaches the same conclusion throughout and is never weaker ($p \leq 0.017$ against every rule-based policy).

Over 28 days the total cost was KRW 10.00 bn without reallocation and KRW 8.52 bn with the proposed policy, a reduction of KRW 1.471 bn (14.7%). The four rule-based policies above use block reallocation only, so this comparison reflects the potential of the architecture as a whole; a comparison matched on action range is treated in Sect. 5.5. The days on which the proposed policy costs more

Table 2: Where the reduction comes from, by demand level (KRW bn, against no reallocation). Columns three to five allow one action at a time; the last column is the share of the total that training adds, the trained model measured against the untrained one. Arrival-time adjustment produces almost all of the reduction and does so on the four congested days, block reallocation helps under light demand but turns negative when demand is highest, and the contribution of training follows the same congested-day pattern.

Daily demand	Daily Days	Both actions	Time only	Block only	Due to training
3,500	12	0.047	-0.013	0.057	0.056
5,000	8	0.060	0.029	0.067	0.029
7,500	4	0.032	0.008	0.000	-0.049
12,500	2	0.611	0.573	-0.014	0.144
15,000	2	0.722	0.747	-0.241	0.495
Total	28	1.471	1.344	-0.131	0.675
<i>share of total</i>		100%	91.4%	—	45.9%

are few and small, whereas the days of largest reduction are those of congested demand.

5.4 Demand Level and the Decomposition of Actions

Table 2 decomposes the reduction by demand level. Of the total, 90.5% occurred on the four days with demand of 12,500 and 15,000 trucks, which supports developing the architecture towards a policy that *intervenes selectively when congestion is observed* rather than continuously. Allowing only one action at a time over the same 28 days separates the two contributions: arrival-time adjustment accounts for 91.4% of the total reduction and almost all of it on those four days, whereas the block-only condition reduces cost under smooth and normal demand (KRW 0.124 bn) but raises it at the highest demand, leaving a net increase of KRW 0.131 bn. The policy can therefore be optimised towards adjusting the relative use of spatial and temporal actions with the demand level. Time-slot quotas were open in this diagnostic run, so the arrival-time figures describe the *range* of what time shifting could provide; a sensitivity analysis with finite quotas would refine the scope of application.

5.5 Rule-Based Policies of the Same Action Range

Sect. 5.4 raises the question of what a rule achieves when given the same action. Two rules were added by transferring the *least-loaded* criterion from the spatial to the temporal axis, introducing no new parameter and sharing the congestion condition of the other rules; Table 1 places them among the rest. Three observations follow. Arrival-time adjustment lowers cost significantly even under a rule (+3.41%, $p = 0.0125$ against no reallocation), so the action itself carries value.

When the action range is matched the proposed policy still costs less than the rule (20/28, $p = 0.036$). Yet on the arrival-time axis alone the *rule was lower on more days* (the proposed policy 8/28) while the total was about four times lower for the proposed policy, because under smooth demand the rule is marginally lower whereas on the congested days the proposed policy is 3.8–5.6 times lower. The learned selection was optimised *not towards winning frequently but towards reducing cost sharply on the expensive days*, and both indicators must be reported for this to be visible. The rule using both actions also weakens under congested demand, reproducing the property of Sect. 5.4—that block reallocation raises cost under congestion—and suggesting it belongs to the action rather than to a particular policy.

5.6 Learning Effect and Dispatching Sensitivity

From KRW 10.00 bn without reallocation, the untrained model reached KRW 9.20 bn and the trained model KRW 8.52 bn, so training adds KRW 0.675 bn, 45.9% of the total. Counting days, however, the trained model was lower on only 18 of 28 ($p = 0.1849$), and the mean daily difference (KRW -24.1 m) is eighteen times the median (-1.3 m). The two statistics disagree because the effect is concentrated rather than spread over the month: the four congested days carry 94.6% of it and the trained model was lower on all four, against 9/12, 5/8, and 0/4 on the lighter days, whereas the 24 remaining days give a mean of -1.5 m with a bootstrap interval that spans zero. Resampling the daily differences within demand strata, which respects the fixed demand mix of the stage, gives a 95% interval of $[-32, -17]$ m per day for the mean difference; resampling without strata widens it to $[-52, -3]$ m, with 99.2% of resamples remaining negative. Both intervals exclude zero, so the direction is consistent, but the narrow one is narrow partly by construction, since each congested stratum holds only two days and resampling within it cannot express uncertainty about how often congestion occurs at all. The conclusion we draw is therefore bounded: learning contributes a reduction that is large in total and without exception on congested days, and counting days understates it because the policy was optimised to cut cost sharply when it is expensive rather than to win frequently—the same property observed for the rules in Sect. 5.5. Establishing it at a conventional significance level requires more congested days, not more days.

Separately, the ranking of crane dispatching rules reversed with demand—the shortest-processing-time rule in current use was 22–25% more expensive than others under smooth and normal demand but the cheapest from 7,500 trucks upward, and lowest overall when weighted by the designed demand mix—so the effect of the reallocation policy should also be re-examined under other dispatching rules.

6 Conclusion

A reinforcement learning architecture that jointly revisits the yard block and arrival time of external truck jobs in response to real-time congestion has been

developed. Proposal and acceptance are modelled as independent policies and a central procedure applies the assignments in one batch after checking mutual consent and resource constraints; the uncertainty in the induced congestion effects is tackled with a counterfactual cost signal from discrete-event simulations that differ in a single action, while at operating time candidates are evaluated by the trained networks alone. Three points follow. Spatial placement and temporal adjustment are compared within one action range. Separating the roles of proposing and accepting extends the logic of distributed appointment negotiation to a real-time reallocation policy [3]. Counterfactual comparison makes the contribution of a single action learnable from a joint cost, evaluating induced waiting and equipment flow in the same monetary unit as the immediate movement cost; because the branch computation is separated from operation, the decision path remains a small cost model and a constraint check. The construction thereby connects arrival-time negotiation [3], yard crane operations [4, 5], and multi-agent counterfactual learning [7, 8] into one spatio-temporal assignment problem.

Observations in the literature-calibrated environment indicate that the structure carries spatio-temporal reallocation through to execution, and that arrival-time adjustment and learned selection produce a direction of cost reduction under congested conditions, which supports the view that real-time information at a smart port can lead to assignment actions rather than monitoring alone [1]. The same direction held when the action range was matched, although on the arrival-time axis alone the rule was lower on more days and the proposed policy led only in total.

6.1 Limitations

Six limitations bound these results. The environment is synthetic: its scale and variation are calibrated to reported ranges, but no operating record of a terminal was fitted, and the demand mix, the arrival-curve coefficients, and two of the four cost coefficients are design values. Congestion is rare on the stage—four of the 28 days—and that is exactly where the effect lives, so the statistics rest on a small number of expensive days; more congested days, rather than more days, is what would settle the learning contribution. The contribution of the consent structure itself has not been isolated: we confirmed that nothing is committed on one-sided consent, but we did not measure what removing the acceptance policy’s veto would cost, so separating proposal from acceptance remains a design choice supported by its operation rather than by an ablation. Time-slot quotas were left open, so the arrival-time results describe an upper range rather than an executable schedule. The three-hour counterfactual horizon is derived from the design—a 30-minute review window, up to 120 minutes of deferral, and a 30-minute arrival-pressure margin—rather than taken from a reported value. Finally, the main comparison uses a single crane dispatching rule, and because the ranking of those rules reverses with demand, the size of the reallocation effect may shift under another one.

Each of these can be addressed one axis at a time: reflecting actual appointment quotas would make the executable range of time adjustment concrete, and repeating a wider variety of congested conditions would delineate the contribution relative to the untrained model. Observation, proposal, acceptance, commitment, execution, and counterfactual learning have been organised into a single reproducible structure, which as automated equipment and appointment information spread can serve as a basis for connecting real-time congestion information to explainable resource assignment decisions.

References

1. Yen, B.T.H., et al.: How smart port design influences port efficiency: a DEA-Tobit approach. *Res. Transp. Bus. Manag.* **46**, 100862 (2023)
2. Chen, G., Govindan, K., Yang, Z.: Managing truck arrivals with time windows to alleviate gate congestion at container terminals. *Int. J. Prod. Econ.* **141**(1), 179–188 (2013)
3. Phan, M.-H., Kim, K.H.: Collaborative truck scheduling and appointments for trucking companies and container terminals. *Transp. Res. B* **86**, 37–50 (2016)
4. Ng, W.C., Mak, K.L.: Yard crane scheduling in port container terminals. *Appl. Math. Model.* **29**(3), 263–276 (2005)
5. Speer, U., Fischer, K.: Scheduling of different automated yard crane systems at container terminals. *Transp. Sci.* **51**(1), 305–324 (2017)
6. Petering, M.E.H., Murty, K.G.: Effect of block length and yard crane deployment systems on overall performance at a seaport container transshipment terminal. *Comput. Oper. Res.* **36**(5), 1711–1725 (2009)
7. Wolpert, D.H., Tumer, K.: Optimal payoff functions for members of collectives. *Adv. Complex Syst.* **4**(2–3), 265–280 (2001)
8. Foerster, J., et al.: Counterfactual multi-agent policy gradients. In: *AAAI*, vol. 32 (2018)
9. Kourounioti, I., Polydoropoulou, A.: Application of aggregate container terminal data for the development of time-of-day models predicting truck arrivals. *EJTIR* **18**(1) (2018)
10. Schwientek, A., Lange, A.-K., Jahn, C.: Simulation-based analysis of dispatching methods on seaport container terminals. *ARGESIM Report* **59**, 357–364 (2020)
11. Wang, W., et al.: Yard crane real-time scheduling among multi-block at terminal: a reinforcement learning based PPO approach. *Transp. Res. E* **207**, 104635 (2026)
12. Kim, S., Mowakeaa, R., Kim, S.-J., Lim, H.: Building energy management for demand response using kernel lifelong learning. *IEEE Access* **8**, 82131–82141 (2020)
13. Kleijnen, J.P.C.: Analyzing simulation experiments with common random numbers. *Manag. Sci.* **34**(1), 65–74 (1988)
14. Ministry of Land, Infrastructure and Transport: Safe trucking freight rates for 2026, Notice 2026-402 (2026). (in Korean)
15. Ministry of Oceans and Fisheries: Port facility charges: berthing rates and calculation basis. (in Korean)